

Final product: Stream Processing Pipeline for Municipal Environmental Sensors

GitHub repository

https://github.com/Morwoi/Projekt_Data_Engineering_DLBDSEDE02

What the repository contains

The repository contains a containerized pipeline comprising Kafka in KRaft mode, MongoDB, a Python producer and a Python consumer. This pipeline streams environmental readings for five stations in Hamburg-Wilhelmsburg from the Open-Meteo API into MongoDB. It also includes host-side scripts for planners and a citizen app backend alongside a sample dataset of real weather from July to September 2026. The README file describes the architecture and configuration in full. The file `producer/stations.json` and the script `docstrings` describe the area and the specific users.

How to use it

Requirements include Docker Desktop or Docker Engine with the Compose plugin. Python 3 with the `pymongo` package is necessary to run the host-side scripts.

1. Start the pipeline

Clone the repository and start the Docker containers.

```
git clone https://github.com/Morwoi/Projekt_Data_Engineering_DLBDSEDE02t
cd Projekt-Data-Engineering-DLBDSEDE02
docker compose up --build -d
```

Readings are automatically stored in the `environment_monitoring.sensor_readings` collection. The Kafka UI is available at `http://localhost:8081`.

2. Check the pipeline's health

Install the required database client and run the status check.

```
pip install pymongo
python scripts/check_status.py
```

3. Load sample data and run planner queries

Populate the database with the sample dataset and execute queries for specific date ranges.

```
python scripts/load_sample_data.py
python scripts/planner_queries.py --from 2026-07-28 --to 2026-07-31
python scripts/planner_queries.py --from 2026-09-15
```

The README file lists further example ranges and details what to look for.

4. Query the citizen-app status

Check the current status for the general public or apply the stricter risk profile for vulnerable people.

```
python scripts/citizen_status.py
python scripts/citizen_status.py --risk-profile vulnerable
```

5. **Verify failover behavior**

Optionally run a script to confirm that the citizen status correctly catches a real outage. This process takes a few minutes.

```
python scripts/verify_citizen_failover.py
```

6. **Stop the environment**

Shut down all containers using the `docker compose down` command. Add the `-v` flag to delete the stored volumes and data.